AI AGENT AS A FIRST-CLASS COLLABORATOR IN THE OPTIMIZATION PLATFORM

Bachelor Thesis

by

Aftab Khan

Advisor:
Dr. Andreas Karrenbauer

April 24, 2026

Contents

I. Introduction **1**
1. Context . . . 1
2. Challenges . . . 2
3. Goal . . . 2
4. Solution Outline . . . 3

II. Related Work **5**
1. LLMs and Mathematical Modeling . . . 5
2. AI Code Generation . . . 6
3. Human-in-the-loop Architecture . . . 7

III. Concept **9**
1. The Collaboration Paradigm . . . 9
2. Schema Validation . . . 10
3. Human-in-the-Loop . . . 10
4. System Workflow . . . 11

IV. Implementation **13**
1. High-Level System Architecture . . . 13
2. Solver-Panel Frontend . . . 14
3. Browser Automation Layer . . . 15
4. LLM Integration . . . 15

V. Conclusion **17**
1. Summary . . . 17
2. Current Status and Next Steps . . . 17

Bibliography **19**

A. An Appendix **21**

Sworn declaration

I declare under oath that I have prepared the paper at hand independently and without the help of others and that I have not used any other sources and resources than the ones stated. Parts that have been taken literally or correspondingly from published or unpublished texts or other sources have been labelled as such. This paper has not been presented to any examination board in the same or similar form before.

I further declare that, where AI-based tools were used during the preparation of this thesis (e.g., for language refinement, brainstorming, or code assistance), their use was critically reviewed and transparently documented where required. The academic content, argumentation, and all final decisions remain my own responsibility.

Saarbrücken, _____ _____

Chapter I.

Introduction

1. Context

1.1. Mixed-Integer Linear Programming

Linear optimization [10] is a method in which a mathematical model formulating a problem is optimized given requirements, constraints, and an objective function in linear form. It is heavily used in mathematics and computer science, but its applications can also be actively seen in real life [9], such as in agriculture for production planning, in industries to maximize efficiency, and in business economics to maximize profit. These are some examples where LP (Linear Programming) is used.

When we talk about linear optimization, it is usually continuous, but not all decisions are continuous in nature. Integer Linear Programming [6] (ILP) includes integer variables which allows for concrete solutions. A mix of both is called Mixed-Integer Linear Programming (MILP), with some discrete and some continuous variables.

1.2. Optimization Platform

Solving such problems mathematically requires expertise, and as we have seen, MILP is widely used in real life; therefore, a more accessible approach to solving such models is warranted. This is where the Optimization Platform [15, 2, 14, 12] comes in. It is a web-based platform used to model and solve MILPs. It provides many features that make the modeling process easier and more accessible, such as a drag-and-drop modeler, the ability to create multiple workspaces to explore different models simultaneously, and the possibility of multi-user collaboration. All of these features share the same goal: making the user's work simpler and more efficient.

1.3. Human-AI Collaboration

However, recent advancements in artificial intelligence, machine learning and the development of large language models [17] have opened the door to promising, previously unseen possibilities for human–machine collaboration. Following our goal of making this

platform more accessible and productive for users, we explore the possibilities of closing the gap between artificial intelligence and the optimization modeler.

2. Challenges

The current feature set of the optimization platform has already made it very accessible and easy to use. However, there are still several aspects that can be improved to take the platform to the next level. Below are some of the existing challenges that this work aims to address:

(i) **Inefficiency:** As the primary method of modeling is via the drag-and-drop modeler interface, relying solely on it for large and complex models can be very time-consuming and redundant. Although this approach makes modeling more intuitive, it significantly decreases efficiency. Increased model complexity also raises the likelihood of errors due to limited supervision and cross-verification.

(ii) **Inaccessibility:** With an increasing number of features on the platform, it becomes difficult for users to familiarize themselves with all available functionalities before they can start building a model. This results in a steep learning curve and a high entry barrier, thereby contributing to the inaccessibility of the platform.

(iii) **Lack of Support:** While human collaborators can join a workspace and work together in teams to support each other, users working independently have no immediate assistance to rely on when they encounter difficulties. Providing live support for such situations would have a positive impact on multiple aspects of the user experience and significantly improve overall usability.

3. Goal

The goal of this work is to integrate an AI agent into the optimization platform so that it can act as a first-class collaborator alongside other users in a shared workspace. The AI agent can be asked, in natural language, to make changes that users want to apply to the current state of the optimization model. The agent then processes the user's request, generates a solution, and applies it to the shared workspace.

After the AI agent applies the changes, they become visible to the users but remain tentative, pending user feedback or approval. Users can review the proposed solution generated by the AI agent and either accept it, reject it, or provide feedback on the generation. This architecture preserves collaboration, human oversight, supervision, and validation.

4. Solution Outline

This section provides a high-level overview of the proposed solution and explains how its individual components work together. We first describe the system flow at an abstract level and then examine each component in greater detail in the subsequent sections.

4.1. AI-Agent Collaborator

A user can begin using the Optimization Modeler by logging in as a master user and creating an optimization model. At this stage, the user can invite an AI agent to the workspace through the same collaboration layer that is already available for human collaborators. The AI agent joins the shared modeling workspace in the same manner as a human user; however, unlike a human collaborator, the AI agent can independently process requests and make modifications to the shared workspace.

4.2. Iterative Feedback

The AI agent is powered by a large language model that interprets user input expressed in natural language and translates it into corresponding changes in the workspace. The user retains full control over this process: when the user issues a request, the AI agent proposes a solution that is displayed in the workspace, after which the user can decide whether to accept or reject the generated changes. The user may also provide additional feedback, which the AI agent uses to iteratively refine and improve its output. This architecture ensures user oversight and control while maintaining a state-of-the-art, stateful, agentic system design.

4.3. Safety and Validation

In addition, a validation layer is applied after the AI model generates its output and before the changes are committed to the workspace. This layer validates the generated model for syntactic and structural correctness, allowing invalid or hallucinated outputs to be discarded. As a result, the validation layer adds an additional safeguard to the overall architecture.

Chapter II.

Related Work

1. LLMs and Mathematical Modeling

1.1. End-to-End Modeler and Solver Archirecture

Extensive research has been conducted at the intersection of Large Language Models (LLMs) and mathematical optimization. A prominent example is **OptLLM** [19], a framework designed to bridge natural language descriptions and specialized solvers. OptLLM utilizes a modular architecture consisting of three primary components: the *Interaction Refinement Module*, which processes user queries and handles multi-round clarifications; the *Converter Module*, which includes a formulator to translate text into mathematical structures and a coder to generate executable code; and finally, the *Responser Module*, which executes the model via external solvers and interprets the results for the user. This architecture demonstrates the feasibility of integrating disparate modules to transform an LLM into a comprehensive end-to-end optimization system.

1.2. Natural Langauge to Formal Specification

Another significant contribution is **LLMOPT** [7], a learning-based framework that focuses on boosting the generalization of LLMs across various optimization types. LLMOPT introduces a universal *Five-Element Formulation* that reduces complex problem descriptions into five core components: sets, parameters, variables, objectives, and constraints. This structured representation allows the system to generate consistent solver code across diverse domains, including linear, nonlinear, and combinatorial optimization.

1.3. LLM Benchmark for Optimization Modeling

While the aforementioned frameworks focus on generation, **Mamo** [5] provides a specialized benchmark to evaluate these modeling capabilities. Mamo consists of 1,209 meticulously curated questions spanning Ordinary Differential Equations (ODEs), Linear Programming (LP), and Mixed-Integer Linear Programming (MILP). Unlike traditional result-oriented metrics, Mamo utilizes a "model generation and numerical validation" approach. An LLM is tasked with formulating a model from the dataset, which is then executed by a solver;

the resulting numerical answer is compared against the provided ground truth. This benchmark is particularly valuable for the evaluation phase of this thesis, as it provides a standardized methodology for measuring the accuracy of AI-generated mathematical models.

2. AI Code Generation

2.1. Current State

Since the introduction of the Transformer architecture [17], there has been a paradigm shift in the field of automated software engineering. While Transformers were initially celebrated for their success in Natural Language Processing (NLP), their ability to model long-range dependencies and complex structural patterns has led to significant advancements in Large Language Models (LLMs) for code generation.

Extensive research has explored the transition from general-purpose text generation to domain-specific code synthesis. As detailed in the comprehensive survey by Jiang et al. [8], modern frameworks leverage massive datasets of source code to perform tasks such as code completion, bug detection, and natural-language-to-code translation. The survey categorizes state-of-the-art models—including GPT-4, CodeLlama, and StarCoder—and examines the underlying training objectives, such as "Fill-in-the-Middle" (FIM) and instruction tuning, which are essential for understanding the non-linear structure of programming languages.

2.2. Coding Agents

Recent advancements in AI-powered coding assistants—such as GitHub Copilot [16], Google Gemini CLI [4], and Anthropic's Claude Code [1]—demonstrate how generative AI significantly enhances software development workflows. These coding agents operate as collaborative tools that integrate directly into development environments, where they utilize Large Language Models (LLMs) fine-tuned on vast code and documentation corpora [18]. Typically, users write natural language prompts or partial code snippets; the agent responds by generating, refactoring, or completing code, suggesting best practices, and occasionally identifying bugs.

Most modern coding agents rely on transformer-based architectures pre-trained on publicly available open-source repositories, and are further tuned for developer intent and task relevance through techniques such as Reinforcement Learning from Human Feedback (RLHF) and tool integration APIs [18]. For example, GitHub Copilot, powered by OpenAI's Codex model, integrates seamlessly into IDEs and proactively suggests entire code blocks in real-time [16]. Google's Gemini CLI and Anthropic's Claude Code Agent similarly offer advanced contextual understanding, enabling cross-file reasoning and multi-modal support such as code explanation, completion, or test synthesis [4, 1].

Despite their power, these tools exhibit key limitations when applied to mathematical optimization modeling. Primarily, their training prioritizes general-purpose software

engineering tasks—such as web development, scripting, or library usage—rather than symbolic reasoning or formal specification needed for optimization problems. As a result, these agents may struggle with generating accurate and semantically valid mathematical models, including algebraic constraints, variable declarations, and objective functions, particularly when such constructs must strictly adhere to domain-specific formats. Furthermore, their validation mechanisms are largely confined to code execution or basic unit tests, whereas optimization modeling requires both syntactic correctness and adherence to deeper mathematical requirements.

Thus, while coding agents have greatly improved code generation and developer productivity in mainstream programming, their architecture and underlying datasets are not inherently optimized for the highly structured, domain-specific tasks characteristic of mathematical optimization modeling.

2.3. LLM Benchmarking for Code Generation

As Large Language Models (LLMs) continue to evolve, the demand for more rigorous evaluation methodologies has led to the development of advanced benchmarking frameworks. One such framework is **EvalPlus** [11], which addresses the limitations of traditional code generation benchmarks.

EvalPlus builds upon established datasets, most notably *HumanEval* [3] and *MBPP*. While these original benchmarks provided a foundational method for measuring functional correctness, their test suites are often sparse and fail to catch edge-case failures, allowing incorrect code to pass as correct. EvalPlus mitigates this by automating the synthesis of thousands of additional test cases for each problem. By significantly increasing the test-case density—often by a factor of 80x compared to the original suites—EvalPlus creates a more robust evaluation environment that exposes "false positives" in LLM-generated code.

3. Human-in-the-loop Architecture

Magentic-UI [13] is a research prototype developed by Microsoft Research that introduces a human-centered AI agent designed to execute complex web-based tasks. Unlike fully autonomous systems, Magentic-UI emphasizes a collaborative relationship between the user and the agent. The system receives natural language queries and interprets user intent to formulate a multi-step plan.

A key contribution of this framework is its implementation of several interaction mechanisms, most notably *co-planning* and *action guards*. During co-planning, the agent reveals its intended steps to the user, who can then audit, alter, or pause the process before execution begins. Furthermore, all critical or irreversible actions—such as submitting sensitive data or performing online transactions—require explicit human approval via action guards. This architecture ensures that operations are conducted within a safe, transparent environment where the user maintains final authority. By prioritizing human

oversight and iterative feedback, Magentic-UI serves as a state-of-the-art example of a well-formed human-in-the-loop (HITL) architecture for web-agentic systems.

Chapter III.

Concept

This chapter presents the detailed conceptual design of the proposed architecture. It explains how the different components of the system interact with each other and how they come together to form a cohesive solution.

1. The Collaboration Paradigm

The Optimization Modeler provides a built-in multi-user collaboration architecture. Our approach builds upon this existing infrastructure in order to preserve modularity and ensure that legacy code remains untouched. In this section, we first describe how collaboration between human users is realized and then explain how the AI agent integrates into the same collaborative workflow.

1.1. Human-to-Human Collaboration

Collaboration between human users is initiated by a master user [15], who registers and logs into the optimization platform hosted at `https://optimized.mpi-inf.mpg.de/modeler/`. After logging in, the master user creates a new modeling workspace and initiates a collaborative session.

To start collaboration, the user clicks the *Share* button located at the bottom of the interface. This action opens a dialog containing shareable links for the current workspace. Two types of links are provided:

- an **edit link**, which grants full editing privileges, and
- a **read link**, which grants view-only access.

Depending on the intended role of the invited participant, the master user shares either the edit link or the read link. This mechanism enables fine-grained access control while maintaining a lightweight collaboration setup.

1.2. AI Agent Integration into the Collaborative Workspace

The AI agent is deployed as a backend service and joins the collaborative workspace in a manner analogous to a human participant. The agent is provided with access to a web browser instance on the server through a web automation framework.

When the AI agent is invited to a workspace, a browser instance is automatically launched on the server. Authentication is performed using a dedicated account that represents the AI agent, after which the edit link is visited to open the shared modeling workspace. As this browser instance runs on the server, the Document Object Model (DOM) of the application can be accessed directly via JavaScript APIs. This enables the AI agent to observe, interact with, and modify the workspace in real time, using the same interface mechanisms available to human users.

2. Schema Validation

The output generated by the large language model (LLM) follows an XML-like syntax that is used internally by the Optimization Modeler. This syntax must conform to a predefined grammar that specifies the allowed constructs, constraints, and structural rules.

One of the major limitations of LLM-based systems is the possibility of hallucinations, meaning that the model may generate syntactically or structurally invalid output. Therefore, before any generated output is applied to the DOM, it must be validated.

To address this, we introduce a validation layer that checks the LLM-generated output against a formally defined grammar. An XML validator is used to verify whether the output conforms to the schema. In the case of validation errors, the error messages are fed back to the model, and the generation process is repeated until a syntactically valid and parsable output is produced. This mechanism prevents invalid or hallucinated outputs from being applied to the workspace.

3. Human-in-the-Loop

Once the LLM-generated output has been validated and applied to the DOM, the user is presented with multiple options. The user may accept the changes and make them permanent, reject the changes—thereby discarding them entirely and reverting to the previous model state—or provide additional feedback through the chat interface.

The AI agent uses this feedback, together with the current state of the optimization model, to iteratively refine its solution. The revised output is then validated and applied to the DOM again. This process continues until the user either accepts or rejects the final solution. This human-in-the-loop mechanism ensures continuous user oversight and provides fine-grained control over the modeling process.

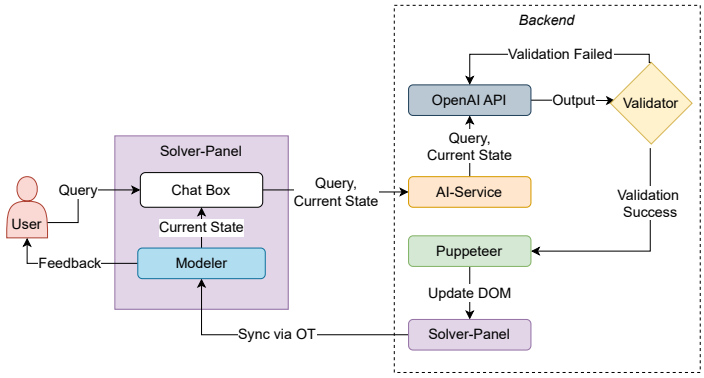

Figure III.1.: Overview of the proposed LLM architecture.

4. System Workflow

The overall system workflow proceeds as follows. Once the AI agent joins the workspace, a chat interface becomes visible to the user, enabling direct communication with the agent. Through this interface, the user can request model modifications, seek assistance, or ask questions related to the optimization model.

Each user request is sent to the AI model along with the current state of the optimization model. The LLM interprets the natural language input and generates the corresponding output. Since the backend has access to the DOM of the modeler instance, the validated output can be applied directly to the workspace.

After the changes are applied, the user observes the updated model in their instance of the Optimization Modeler. The human-in-the-loop mechanism then allows the user to further refine, accept, or reject the generated changes before they are finalized.

Chapter IV.

Implementation

In this chapter, we move from the conceptual design to the concrete implementation of the proposed architecture. We discuss the technologies and frameworks employed, how the individual components are interconnected, and the design decisions made during implementation.

1. High-Level System Architecture

1.1. Overview

The primary access point of the Optimization Modeler platform is the **Solver-Panel**, which is implemented as an Angular application. It is responsible for core functionalities such as user authentication, model creation and editing, and multi-user collaboration. The Solver-Panel communicates with the backend service introduced in this work, which hosts the AI agent and the auxiliary services required to support the proposed architecture.

User interaction with the AI agent takes place through a chat interface embedded within the Angular application. When the user submits a prompt via this chat interface, the request is sent to the backend API service using HTTP-based communication.

1.2. Class Hierarchy

We now describe the concrete classes involved in the architecture. The **Solver-Panel** class provides the frontend interface of the Optimization Modeler. On the backend, we implement an Express.js-based service referred to as the **AI-Agent** service. This service consists of two main components. The first is the **LLM** service, which provides an abstraction layer for interacting with large language models—in our case, OpenAI's GPT-based models—through their API. The second component is the **Puppeteer-Controller**, which is responsible for controlling a server-side browser instance using the Puppeteer automation framework. This controller enables the AI agent to interact with the Optimization Modeler by programmatically accessing and manipulating the Document Object Model (DOM).

2. Solver-Panel Frontend

This section describes the user-facing component of the Optimization Platform, referred to as the *Solver-Panel*, and its integration with the AI agent service.

2.1. User Authentication

Users begin by creating an account and authenticating via the Solver-Panel's dedicated security layer. This infrastructure manages user sessions and access control for various optimization workspaces. The underlying authentication workflows and the management of user identities are implemented following the architecture established by Begaj [2].

2.2. Internal Modeler

The core of the interface is the Optimization Modeler, a vanilla JavaScript module integrated into the Angular-based Solver-Panel application. Once authenticated, users can construct Mixed-Integer Linear Programs (MILPs) using an intuitive drag-and-drop interface. This visual representation is backed by a domain-specific, XML-like markup language. Any operation performed within the graphical Document Object Model (DOM) is automatically translated into this syntax, which serves as the source of truth for the optimization model [14, 12].

2.3. Collaboration and Agent Invitation

The platform supports multi-user collaboration through a link-sharing architecture. Users can generate and share session links that allow others to join a workspace with either read-only or editing privileges [15]. To facilitate AI assistance, we extended this mechanism to include an "Invite AI Agent" functionality.

When triggered, the frontend sends an HTTP POST request to the AI agent service's `invokeai` (or `/puppet`) endpoint. This request includes the `writeUrl` (edit link) of the current session. The backend server uses this link to launch a headless browser instance that connects to the workspace, effectively joining the session as a virtual collaborator.

2.4. Chat Interface and Request Processing

To enable natural language interaction, the modeler includes a dedicated chat interface. When a user submits a query, the frontend invokes the agent's `processrequest` (or `/openai`) API endpoint. The payload for this request consists of two primary components: the current state of the optimization model (extracted in its XML format) and the user's natural language prompt. The agent service then leverages a Large Language Model to interpret the request within the context of the existing model, generating the necessary modifications which are subsequently applied back to the workspace.

3. Browser Automation Layer

3.1. Initialization

When the API endpoint `invokeAI` is called, the `puppetInit()` method of the Puppeteer controller is executed first. This method launches an instance of a headless browser (Google Chrome) on the server, navigates to the login page of the Optimization Modeler, enters the authentication credentials associated with the AI user, and completes the login process.

After the AI user has successfully authenticated, Puppeteer is used again to navigate to the `editURL` received in the response body. Visiting this URL opens the shared instance of the optimization model within the browser. Since this browser instance runs on the backend server, the shared modeler instance can now be accessed and manipulated programmatically through the backend.

3.2. DOM Manipulation

This layer is responsible for handling DOM manipulation by leveraging the capabilities provided by the Puppeteer service. After the user submits a request through the chat interface and the LLM service generates the corresponding output, the backend accesses the `innerHTML` of the modeler instance's document body and replaces the current model with the model generated by the LLM.

This operation triggers the same internal update mechanisms as those invoked when a human user manually modifies the model through the interface. As a result, the change is propagated automatically to all users connected to the same collaborative session, ensuring consistency across all active instances.

4. LLM Integration

4.1. Context

When a user submits a request through the chat interface, the `/openaiResponse` endpoint is invoked. This endpoint receives the current state of the optimization model along with the user's prompt in the request body. The backend then forwards this information to the LLM as contextual input.

Providing the current state of the model allows the LLM to understand what the user is currently working on and the progress made so far. The quality and usefulness of the LLM's output are strongly influenced by the clarity and specificity of the user prompt; consequently, more detailed prompts generally result in more accurate and helpful responses.

4.2. Prompt Engineering

Prompt engineering plays a crucial role in generating functional and reliable outputs from large language models. While users are encouraged to formulate prompts that clearly and precisely describe their requirements, system-level prompts must also be carefully designed.

The LLM must be explicitly instructed about its role and responsibilities. In our case, the model operates as part of an AI agent whose purpose is to assist users in creating and modifying optimization models. The system prompt therefore includes a clear description of this role, as well as the current model state and the user's request.

In addition, the prompt may include one or more illustrative examples that demonstrate the expected input–output behavior. This approach leverages the few-shot learning capabilities of LLMs, enabling the model to better align its responses with the desired format and semantics.

Chapter V.

Conclusion

1. Summary

This thesis has presented a novel architecture for integrating an AI agent as a "first-class collaborator" within a web-based Optimization Platform. By bridging the gap between natural language processing and visual modeling, the proposed system addresses the core challenges of inefficiency and high entry barriers in Mixed-Integer Linear Programming (MILP).

The primary technical contribution of this work is the "Phantom User" paradigm. By utilizing a headless browser automation layer (Puppeteer), the AI agent is able to join collaborative modeling sessions as a virtual participant. This approach leverages the platform's existing Operational Transformation (OT) infrastructure, ensuring that AI-driven modifications are synchronized in real-time across all connected clients without requiring invasive changes to the legacy backend.

To ensure the reliability of the system, we implemented a multi-layered safety framework. A dedicated validation layer enforces syntactic and structural correctness of the LLM-generated XML-like markup, effectively mitigating the risk of model corruption due to hallucinations. Furthermore, a stateful "Human-in-the-Loop" workflow ensures that the user remains the ultimate authority, providing a mechanism for iterative feedback and final approval of all agent-led changes.

In conclusion, this work demonstrates that AI agents can move beyond being simple "tools" to become active, collaborative participants in complex technical domains, significantly enhancing the accessibility and productivity of optimization modeling.

2. Current Status and Next Steps

While the theoretical architecture has been established, the implementation is currently in a functional prototype stage. This section outlines the current capabilities and the prioritized milestones for future development.

2.1. Implementation Status

The core communication pipeline between the *Solver-Panel* frontend and the *AI-Agent* backend service is fully operational. A dedicated chat interface has been integrated into the modeler, enabling natural language interaction. Currently, the backend service successfully utilizes Puppeteer to join collaborative sessions and the OpenAI API to translate user prompts into model updates. The agent is capable of perceiving the model state, generating modifications, and applying them directly to the Document Object Model (DOM) of the shared workspace.

2.2. Future Milestones

To reach the final vision of a robust and secure collaborative environment, the following components are currently under development:

- **Human-in-the-Loop Implementation:** The current version applies changes directly to the model. The next phase involves implementing the full iterative feedback loop (Accept/Reject/Refine) using a stateful framework like LangGraph. This will ensure that all AI-driven modifications remain tentative until they receive explicit user approval.
- **Schema-Based Validation:** Work is ongoing to define a formal grammar for the platform's XML-like syntax. This schema will form the basis of the validation layer, allowing the agent to detect and correct "hallucinations" or syntax errors before they are presented to the user.
- **UI/UX Enhancements:** To improve the collaborative experience, further UI features will be added to provide the user with clear visual cues regarding the agent's current state (e.g., "Thinking," "Validating," or "Awaiting Review").

Bibliography

[1] Anthropic. *Claude Code Agent: A Code Assistant for Claude*. 2025. URL: https://code.claude.com/docs/en/overview.

[2] Deni Begaj. *A multi-user platform for managingand solving Mixed Integer Linear Optimization problems*. Master Thesis at Saarland University. 2020.

[3] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG]. URL: https://arxiv.org/abs/2107.03374.

[4] Google. *Gemini CLI: A Command-Line Interface for Gemini*. 2025. URL: https://geminicli.com/.

[5] Xuhan Huang et al. *LLMs for Mathematical Modeling: Towards Bridging the Gap between Natural and Mathematical Languages*. 2025. arXiv: 2405.13144 [cs.AI]. URL: https://arxiv.org/abs/2405.13144.

[6] *Integer programming — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Integer%20programming&oldid=1325150256. 2026.

[7] Caigao Jiang et al. *LLMOPT: Learning to Define and Solve General Optimization Problems from Scratch*. 2025. arXiv: 2410.13213 [cs.AI]. URL: https://arxiv.org/abs/2410.13213.

[8] Juyong Jiang et al. "A Survey on Large Language Models for Code Generation". In: *ACM Transactions on Software Engineering and Methodology* 35.2 (Jan. 2026), pp. 1–72. ISSN: 1557-7392. DOI: 10.1145/3747588. URL: http://dx.doi.org/10.1145/3747588.

[9] Rajendra Kunwar and HP Sapkota. "An introduction to linear programming problems with some real-life applications". In: *European Journal of Mathematics and Statistics* 3.2 (2022), pp. 21–27.

[10] *Linear programming — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Linear%20programming&oldid=1329325329. 2026.

[11] Jiawei Liu et al. *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation*. 2023. arXiv: 2305.01210 [cs.SE]. URL: https://arxiv.org/abs/2305.01210.

[12] Maksim Moor. *From No-Code to Low-Code: Augmenting Visual Modeling for Mixed-Integer Optimization*. Bachelor Thesis at Saarland University. 2024.

[13] Hussein Mozannar et al. *Magentic-UI: Towards Human-in-the-loop Agentic Systems*. 2025. arXiv: 2507.22358 [cs.AI]. URL: https://arxiv.org/abs/2507.22358.

[14] Marco Mrsic. *An interface to model and solve Mixed-Integer Linear Programs*. Bachelor Thesis at Saarland University. 2020.

[15] Marco Mrsic. *Real-Time Collaborative Optimization Platform*. Master Thesis at Saarland University. 2023.

[16] OpenAI. *Copilot: A Code Assistant for Copilot*. 2025. URL: https://github.com/features/copilot.

[17] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[18] Jian Yang et al. *From Code Foundation Models to Agents and Applications: A Comprehensive Survey and Practical Guide to Code Intelligence*. 2025. arXiv: 2511.18538 [cs.SE]. URL: https://arxiv.org/abs/2511.18538.

[19] Jihai Zhang et al. *Solving General Natural-Language-Description Optimization Problems with Large Language Models*. 2024. arXiv: 2407.07924 [math.OC]. URL: https://arxiv.org/abs/2407.07924.

Appendix A.

An Appendix